MyCodeSchool

exercițiu 04 — încapsularea datelor

Rândul din SAP devine un obiect, testul compară date · Playwright / TypeScript · Andreea

Unde lucrezi: `sap-starter/`, peste rezolvarea ta de la exercițiul 03.
Nou, ai voie: `interface` în `models/`, o metodă care întoarce un obiect (`Promise<FrParentRow>`), `toEqual` pe obiecte.
Regula rămâne: niciun `expect` în `pages/`. Pagina citește, testul verifică.
Înainte să începi: `npx tsc --noEmit` fără erori și `173 passed`. La final, aceleași cifre.

Partea 1 — FR_PARENT

1. Tipul rândului

nivel 1

Fișier nou: `models/FrParentRow.ts`.

```
export interface FrParentRow {
  esid: string;
  frGuid: string;
  entGuid: string;
  status: string;
}
```

Patru coloane din cinci: `DESCRIPTION` e text pe mai multe rânduri și nu o verificăm aici.

Teorie: fișa `exercitii-abstractizare`, exercițiul cu `interface Exportabil` — o interfață spune ce formă are un obiect, fără cod.

Gata când: tsc nu dă nicio eroare.

2. readRow în FrParentPage

nivel 1

Fișierul `pages/FrParentPage.ts`, sub constructor. Importă `type { FrParentRow }` din `../models/FrParentRow.js`.

1. Semnătura: `async readRow(rowIndex: number): Promise<FrParentRow>`
2. Întoarce un obiect cu cele 4 câmpuri; fiecare valoare e `await this.getCellText(rowIndex, "<COLOANA>")`, cu `ESID`, `FR_GUID`, `ENT_GUID`, `STATUS`.

Indiciu: `return { esid: await this.getCellText(rowIndex, "ESID"), ... };` — numele coloanelor din SAP apar *doar aici*, nicăieri în test.

Gata când: tsc nu dă nicio eroare.

De reținut: ce e în interiorul rândului (celule, `data-column-name`, `innerText`) e treaba paginii. Testul primește doar date.

3. single row cu un singur expect

mai greu

Fișierul `tests/fr-parent/fr_parent.spec.ts`, blocul `Middle-Parametrizabil`, cazurile `singleRowByEsid`.

1. Tipul cazurilor: `{ esid: string; expected: FrParentRow }[]` (importă `type { FrParentRow }`).
2. Fiecare `expected` are acum toate cele 4 câmpuri: `esid`, `frGuid`, `entGuid` (din ancore, ca până acum) și `status`: `"Active"` pentru ESID-777, `"Pending"` pentru ESID-778, `"Blocked"` pentru ESID-779.
3. În test, bucla `for (const [column, value] of ...)` se înlocuiește cu un singur rând.

Înainte / după:

```
for (const [column, value] of Object.entries(testCase.expected)) {
  await expect(frParentPage.tableCell(0, column)).toHaveText(value);
}
```

```
expect(await frParentPage.readRow(0)).toEqual(testCase.expected);
```

Rezultat așteptat: `173 passed`.

Indiciu: lasă pe loc `expect.poll(() => frParentPage.getRowCount()).toBe(1)` dinaintea lui `readRow`: el așteaptă să se termine filtrarea. Fără el, `readRow(0)` poate citi rândul de dinainte de filtru.

Gata când: `173 passed` și în test nu mai apare niciun nume de coloană.

Partea 2 — ITM_DETAILS și dovada

4. Același lucru pentru ITM_DETAILS

mai greu

1. `models/ItmDetailsRow.ts`: `parentId`, `callOff`, `setupFr`, `region`, toate `string`.
2. `ItmDetailsPage.readRow(rowIndex)`, din coloanele `PARENT_ID`, `CALLOFF`, `SETUP_FR`, `REGION`.
3. În `tests/itm-details/item-details.spec.ts`, testul parametrizat `4,7 ... CALL-OFF`: primește `{ itmDetailsPage }` în loc de `{ sapPage }`, execută cu `itmDetailsPage.executeTable(itmDetailsPage.tableName, c.filters)`, apoi `const row = await itmDetailsPage.readRow(0);`
4. Cele 4 `expect`-uri pe celule devin unul: `expect(row).toEqual({ parentId: c.expected.parentId, ... })` cu cele 4 câmpuri din `c.expected`.
5. Al cincilea rămâne, pe obiect: `expect(row.setupFr).toContain(c.expected.frGuid);`

Rezultat așteptat: `173 passed`.

Gata când: `173 passed` și testul nu mai are niciun `sapPage.cell(...)`.

5. Ce vezi când datele sunt greșite

mai greu

1. În cazurile de la exercițiul 3, schimbă `"Pending"` în `"Pendin"` și rulează `npx playwright test tests/fr-parent --project=chromium`. Pune-l la loc.

Rezultat așteptat: un test pică, iar diff-ul arată tot rândul, cu câmpul greșit marcat:

```
-   "status": "Pendin",
+   "status": "Pending",
```

Întrebare de răspuns în scris: în SAP, coloana `STATUS` se redenumește în `STATE`. Câte fișiere modifici acum? Câte ar fi trebuit modificate înainte de fișa asta?

Gata când: ai văzut diff-ul, totul e la loc (`173 passed`) și ai răspuns la întrebare.

De reținut: un test de date compară *rânduri*, nu celule. Când pică, îți arată exact ce valoare din SAP e diferită.

Partea 3 — readRow pe toate paginile

Ce ai primit: după `git pull` ai 4 teste noi, scrise *celulă cu celulă*, ca înainte de fișa asta: `tests/entitlement/`, `tests/fr-audit/`, `tests/mailcount/`, `tests/case-notes/` (9 teste).
Ce faci: le refactorizezi ca la exercițiile 1–4: tip în `models/`, `readRow` în pagină, un singur `toEqual` în test.
Înainte și după fiecare exercițiu: `npx tsc --noEmit` fără erori și `npx playwright test tests/<folder> --project=chromium` cu același număr de teste verzi.

6. ENTITLEMENT

nivel 1

1. `models/EntitlementRow.ts`: `entGuid`, `entType`, `active`, `createdAt`, toate `string`.
2. `EntitlementPage.readRow(rowIndex)`, din coloanele `ENT_GUID`, `ENT_TYPE`, `ACTIVE`, `CREATED_AT`.
3. În `tests/entitlement/entitlement.spec.ts`, tipul cazurilor devine `{ title: string; filters: Record<string, string>; expected: EntitlementRow }[]`, iar cheile din `expected` se scriu ca în interfață (`ENT_TYPE` → `entType`).
4. Cele 4 `expect`-uri pe celule devin unul: `expect(await entitlementPage.readRow(0)).toEqual(c.expected);`

Rezultat așteptat: `2 passed` în `tests/entitlement`.

Gata când: `2 passed` și în test nu mai apare niciun nume de coloană.

7. FR_AUDIT

mai greu

Același refactoring pe `tests/fr-audit/fr-audit.spec.ts`: `FrAuditRow` cu 5 câmpuri, din `AUDIT_ID`, `FR_GUID`, `EVENT_TYPE`, `CHANGED_BY`, `CHANGED_AT`. `DETAILS` rămâne pe dinafară, ca `DESCRIPTION` la FR_PARENT: e text pe mai multe rânduri.

Gata când: `2 passed` în `tests/fr-audit` și un singur `expect` pe rând.

8. MAILCOUNT

mai greu

Refactoring pe `tests/mailcount/mailcount.spec.ts`: `MailCountRow` cu `mailId`, `parentId`, `count`, `owner`, `status`.

Indiciu: `getCellText` întoarce `string`. Dacă scrii `count: number`, `tsc` te oprește în `readRow`. În tabel vezi `3`, dar în test compari `"3"`.

Gata când: `3 passed` în `tests/mailcount`.

9. CASE_NOTES

provocare

Refactorizează singură `tests/case-notes/case-notes.spec.ts`, fără `NOTE_TEXT`.

Gata când: `2 passed` în `tests/case-notes`, iar în test nu mai apare niciun `tableCell`.

Partea 4 — orice tabel știe să-și citească rândul

10. readRow devine obligatoriu

provocare

1. În `TablePage`, sub `columns`:

   ```
   abstract readRow(rowIndex: number): Promise<object>;
   ```

2. În `tests/tables/tables.spec.ts`, în buclă, după `expect.poll(...)`: citește rândul 0 al fiecărui tabel și verifică că nicio valoare din el nu e goală (`Object.values(...)` + `not.toContain("")`).
3. Dovada: comentează `readRow` din `CaseNotesPage` și rulează `npx tsc --noEmit`. Pune-l la loc.

Rezultat așteptat la pasul 3:

```
error TS2515: Non-abstract class 'CaseNotesPage' does not
implement inherited abstract member readRow from class 'TablePage'.
```

Întrebare de răspuns în scris: de ce poate bucla din `tables.spec.ts` să cheme `readRow(0)` pe un `TablePage`, deși fiecare pagină întoarce alt tip de rând?

Gata când: `tsc` fără erori, `npx playwright test --project=chromium` are `180 passed` și ai răspuns la întrebare.

De reținut: `abstract` e o regulă verificată de `tsc`: un tabel nou (al 7-lea) nu compilează până nu-și scrie `readRow`.